

ASG AIRLINES
END-TO-END DATA ENGINEERING CASE
STUDY

INTERNSHIP PROJECT DOCUMENTATION

Submitted by

Divya Dharshini

VIT Chennai

Table of Contents

1. Project Overview
2. Problem Statement
3. Project Objectives
4. Dataset Description
5. Tools and Technologies
6. Solution Architecture
7. Data Flow Diagram
8. Data Model
9. Data Ingestion
10. Data Cleaning and Quality Checks
11. Data Transformation
12. Flight Duration Calculation
13. Overnight and Cross-Day Flight Handling
14. Validation and Error Handling
15. Business KPIs
16. Power BI Dashboard and Reporting
17. Assumptions
18. Data Privacy and Governance
19. End-to-End Workflow
20. Key Outcomes and Insights
21. Conclusion
22. Dashboard Screenshot and Visual Interpretation

1. Project Overview

The ASG Airlines case study focuses on building a reliable end-to-end data engineering pipeline for airline flight data. The main purpose of the project is to take raw flight records, validate and clean them, standardize important fields, calculate useful flight-level measures, and prepare the resulting data for business reporting.

The completed implementation was developed in Google Colab using Python-based data processing. The notebook was used as the implementation layer for data ingestion, data quality checks, cleaning, transformation, and KPI preparation. The final cleaned data can then be consumed by a reporting layer such as Power BI for interactive analysis.

The assignment specifically highlights corrupted flight identifiers, inconsistent time formats, missing values, and overnight or cross-day flights as important data quality challenges. The documentation therefore explains how the implemented pipeline addresses these areas and how the processed data supports airline performance analysis.

2. Problem Statement

Airline operational data may contain inconsistent or incomplete records that make direct analysis unreliable. In this case study, the raw flight dataset contains fields such as flight ID, airline, source, destination, departure time, and arrival time. These fields must be standardized before business metrics can be calculated correctly.

A major challenge is time handling. A flight can depart late in the day and arrive after midnight on the following calendar day. If the arrival time is treated as belonging to the same day without considering this situation, the calculated duration can become negative or incorrect. Therefore, the pipeline must account for cross-day flights.

The solution also needs to identify duplicates, missing values, invalid records, and schema or formatting inconsistencies. After cleaning, the data should support KPIs such as average flight duration, route-wise traffic, airline distribution, and delay or anomaly analysis. The assignment also expects an interactive Power BI reporting layer.

3. Project Objectives

The main objective is to create a standardized and analysis-ready airline dataset through a repeatable data engineering workflow. The implementation is designed to move from raw input data to cleaned and transformed records and finally to business-oriented reporting.

- Load the source airline dataset into the Python processing environment.
- Inspect the dataset structure, data types, missing values, and duplicate records.
- Clean inconsistent or invalid records and standardize relevant fields.
- Convert time information into a consistent representation.
- Calculate flight duration correctly, including overnight or cross-day flights.
- Prepare reliable data for KPI calculation and Power BI reporting.
- Document assumptions, validation logic, transformations, and reporting outputs.

The project also emphasizes reliability and traceability. Each transformation should have a clear business reason so that the final KPI values can be explained from the original source records.

4. Dataset Description

The case study defines the core airline data around six fields: flight_id, airline, source, destination, departure_time, and arrival_time. These fields describe the flight identifier, operating airline, origin, destination, departure time, and arrival time.

The dataset is the primary input to the implemented notebook. The first stage of the pipeline reads the source file and inspects its structure before applying transformations. This approach helps prevent calculations from being performed on records that have obvious formatting or quality problems.

The flight_id field is useful for identifying individual flight records. The airline field supports airline-level comparisons. Source and destination support route analysis. Departure and arrival time fields are required for duration calculations and for identifying unusual or inconsistent time relationships.

This table represent the Airline Dataset

Field	Description
flight_id	Unique flight identifier
Airline	Name of the operating airline
Source	Departure location
Destination	Arrival location
departure_time	Flight departure timestamp
arrival_time	Flight arrival timestamp
duration	Existing flight duration

5. Tools and Technologies

The implementation uses Python in Google Colab as the main development environment. Python provides a practical way to automate ingestion, cleaning, transformation, validation, and KPI preparation in one reproducible notebook.

- Python for the overall data processing workflow.
- Pandas for tabular data loading, cleaning, transformation, grouping, and export.
- NumPy where numerical processing is required.
- Date and time functions for standardizing and calculating flight timings.
- Matplotlib or other notebook visualization functionality where used in the implementation for checking data patterns.
- Power BI as the intended reporting and dashboard layer for interactive business analysis.

The exact libraries and functions used in the notebook form the implementation reference for this documentation. The report does not replace the notebook; instead, it explains the processing logic and how the outputs are intended to be used.

Technologies used:

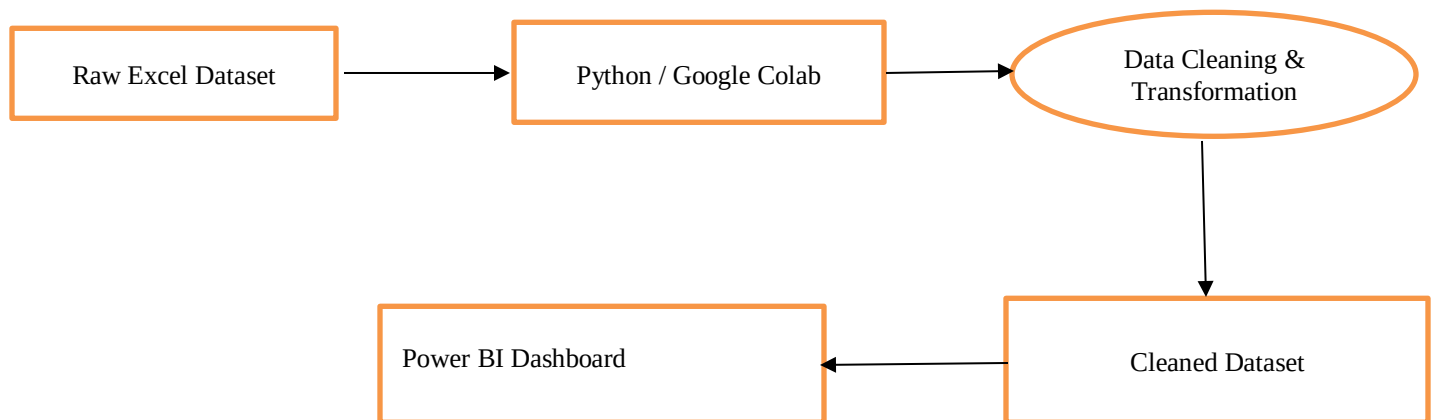
- Python
- Pandas
- NumPy
- DateTime processing
- Matplotlib where required for analysis
- Power BI
- Excel

6. Solution Architecture

The solution follows a simple batch data engineering architecture. The source airline file is ingested into the Python environment, processed through a sequence of quality and transformation steps, and then prepared as a cleaned dataset for reporting.

At a high level, the architecture contains four logical layers. The source layer contains the raw Excel dataset. The processing layer contains the Google Colab Python notebook. The curated data layer contains the cleaned and transformed airline records. The reporting layer uses the curated data to create Power BI visuals and KPIs.

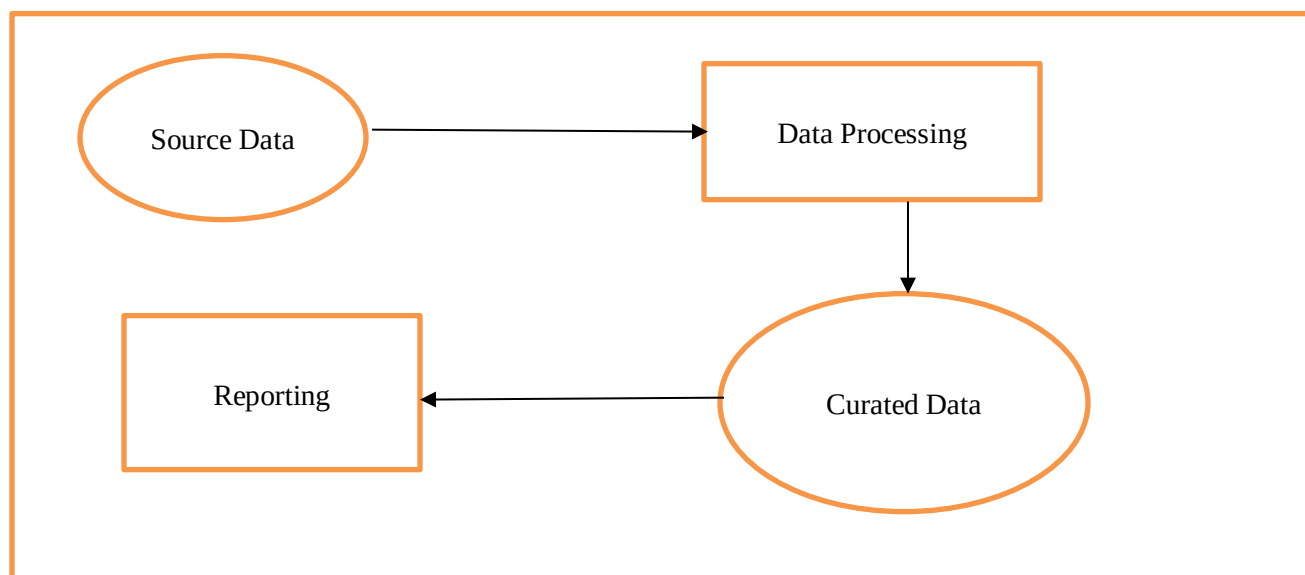
This design is suitable for a case study because each stage has a clear responsibility. Separating ingestion, cleaning, transformation, and reporting also makes the workflow easier to test and extend later. If the data source changes in the future, the ingestion stage can be updated without redesigning the entire reporting concept.



7. Data Flow Diagram

The data flow starts when the raw airline source file is loaded into the notebook. The raw records are then inspected for schema, missing values, duplicates, and invalid time information. The cleaning stage produces standardized records, after which transformation logic creates analytical fields such as flight duration.

The transformed data is then used for KPI calculations and dashboard preparation. The flow is therefore directional: source data enters the pipeline, quality checks and transformations are applied, and the resulting trusted dataset moves into reporting.

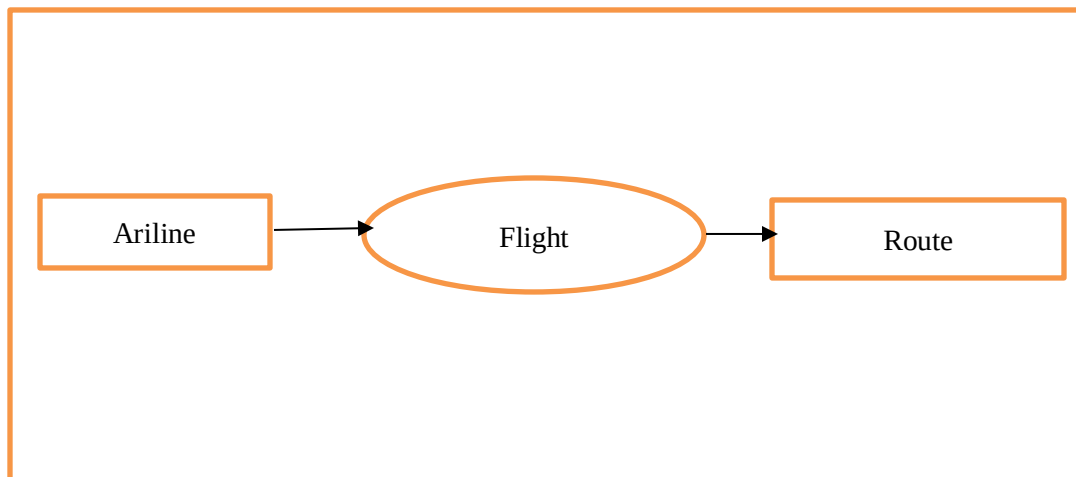


8. Data Model

The analytical model is centered on the flight record because the supplied case study is primarily flight-level data. The main attributes are flight_id, airline, source, destination, departure_time, arrival_time, and derived duration information.

For dashboard analysis, the flight table can act as the central analytical table. Airline and route attributes are used to group records, while timing fields and derived measures support duration and anomaly analysis. If the Power BI implementation uses additional KPI or helper tables, those tables should be treated as reporting structures rather than replacements for the cleaned flight records.

A simple logical model is therefore: one flight record contains one airline, one source, one destination, one departure time, and one arrival time. Derived duration and anomaly indicators are calculated from the standardized time fields.



9. Data Ingestion

The ingestion stage reads the source airline dataset into a Pandas DataFrame. The notebook uses file-based ingestion suitable for the provided case study. The purpose of this stage is to bring the raw data into a structured form while preserving the source columns needed for later validation and transformation.

Code:

```
#Load all sheets from the Excel file

file_path = "/content/UseCase - Airlines.xlsx"

flights = pd.read_excel(file_path, sheet_name="flights")
bookings = pd.read_excel(file_path, sheet_name="bookings")
payments = pd.read_excel(file_path, sheet_name="payments")
passengers = pd.read_excel(file_path, sheet_name="passengers")

print("Data loaded successfully!")
print()
print("Flights:", flights.shape)
print("Bookings:", bookings.shape)
print("Payments:", payments.shape)
print("Passengers:", passengers.shape)
```

After loading, the data should be inspected using row counts, column names, data types, and sample records. This establishes the baseline dataset before any transformations are performed. Keeping the raw ingestion step separate from cleaning also makes the pipeline easier to reproduce.

Output:

```
Data loaded successfully!

Flights: (1020, 7)
Bookings: (1000, 9)
Payments: (1000, 4)
Passengers: (1039, 9)
```

10. Data Cleaning and Quality Checks

The cleaning stage focuses on making the dataset consistent and usable for analysis. Duplicate records can cause flight counts and KPI values to be overstated, so duplicate handling is an important quality check. Missing values also need to be identified because missing identifiers, routes, or timings can prevent correct downstream calculations.

Code:

```
#Remove exact duplicate rows

flights = flights.drop_duplicates().reset_index(drop=True)
bookings = bookings.drop_duplicates().reset_index(drop=True)
payments = payments.drop_duplicates().reset_index(drop=True)
passengers = passengers.drop_duplicates().reset_index(drop=True)

print("Duplicate rows after cleaning:")
print("Flights:", flights.duplicated().sum())
print("Bookings:", bookings.duplicated().sum())
print("Payments:", payments.duplicated().sum())
print("Passengers:", passengers.duplicated().sum())
```

The notebook's cleaning logic should be treated as the authoritative implementation. The key principle is that a record should not silently produce an invalid KPI. Records that cannot support a reliable calculation should be identified through validation rules rather than allowing incorrect values to flow into the dashboard.

Output:

```
Data loaded successfully!

Flights: (1020, 7)
Bookings: (1000, 9)
Payments: (1000, 4)
Passengers: (1039, 9)
```

11. Data Transformation

Transformation converts the cleaned raw fields into standardized analytical fields. Time columns are converted into a consistent datetime representation so that subtraction and comparison operations behave predictably.

```
#Remove exact duplicate rows

flights = flights.drop_duplicates().reset_index(drop=True)
bookings = bookings.drop_duplicates().reset_index(drop=True)
payments = payments.drop_duplicates().reset_index(drop=True)
passengers = passengers.drop_duplicates().reset_index(drop=True)

print("Duplicate rows after cleaning:")
print("Flights:", flights.duplicated().sum())
print("Bookings:", bookings.duplicated().sum())
print("Payments:", payments.duplicated().sum())
print("Passengers:", passengers.duplicated().sum())
```

Additional transformations can include standardized categorical values, derived date components, and analytical flags. These transformations are important because dashboards depend on consistent dimensions. For example, route analysis requires source and destination values to be consistently represented.

The transformation stage should be deterministic: running the same source data through the same notebook logic should produce the same cleaned and transformed result.

12. Flight Duration Calculation

Flight duration is one of the most important derived measures in the case study. It is calculated from standardized arrival and departure timestamps. The general calculation is arrival timestamp minus departure timestamp.

Code:

```
#Compare existing duration with calculated duration

flights['duration'] = pd.to_timedelta(flights['duration'].astype(str), errors='coerce')

flights["existing_duration_hours"] = (
    flights["duration"].dt.total_seconds() / 3600)

flights["duration_difference"] = (
    flights["existing_duration_hours"]
    - flights["calculated_duration_hours"])

display(
    flights[[
        "flight_id",|
        "existing_duration_hours",
        "calculated_duration_hours",
        "duration_difference"]].head(10))
```

The result can be stored as a duration value and then converted to minutes or hours for KPI reporting. A consistent unit is important because average duration and route comparisons should use the same measurement across all records.

Duration values should also be checked for unrealistic negative or extremely large results. Such values may indicate a source-data problem, incorrect time interpretation, or an unhandled cross-day flight.

13. Overnight and Cross-Day Flight Handling

An overnight flight is a flight where the arrival time occurs after midnight on the following calendar day. A simple time-only subtraction can incorrectly interpret such a flight as arriving before it departed. The pipeline therefore needs logic that recognizes when the arrival clock time is earlier than the departure clock time and adjusts the arrival date accordingly.

Conceptually, if a flight departs at a late evening time and arrives after midnight, the arrival timestamp belongs to the next day. Adding one calendar day to the arrival timestamp before subtraction produces the correct positive duration.

This handling is important for both accuracy and anomaly detection. Without it, overnight routes could be incorrectly classified as negative-duration anomalies and could distort average flight duration calculations.

14. Validation and Error Handling

Validation is performed to ensure that the transformed dataset is suitable for reporting. Important checks include whether required columns exist, whether time fields can be parsed, whether duplicate records remain, whether missing critical values remain, and whether calculated durations are logically valid.

- Confirm that the expected source columns are available.
- Check missing values before and after cleaning.
- Check duplicate records before and after duplicate removal.
- Verify that standardized time fields can be used for arithmetic.
- Identify negative or otherwise suspicious duration values.
- Review row counts after major transformations to detect unexpected data loss.

These checks provide a basic data-quality layer. In a production pipeline, the same principles could be expanded into automated validation rules, logging, rejected-record storage, monitoring, and alerting.

15. Business KPIs

The assignment requires business-oriented KPIs that translate cleaned flight records into useful airline insights. The core KPIs include average flight duration, route-wise traffic, delays or anomalies, and distribution of flights by airline.

- Average Flight Duration: average duration across valid flight records.
- Route-wise Traffic: number of flights grouped by source and destination.
- Delays / Anomalies: identification of unusual or invalid timing patterns based on the available fields and implemented logic.
- Flights by Airline: distribution or count of flights for each airline.
- Additional analytical measures can be derived from the cleaned dataset when supported by the available columns.

KPI calculations should be performed only after cleaning and standardization. This ensures that duplicate records, malformed times, and invalid durations do not unnecessarily distort the business results.

Code:

```
#Airline KPI

airline_kpi = flights.groupby("airline").agg(
    Total_Flights=("flight_id", "count"),
    Average_Duration_Hours=("flight_duration_hours", "mean")).reset_index()
airline_kpi["Average_Duration_Hours"] = (airline_kpi["Average_Duration_Hours"].round(2))
display(airline_kpi)
```

16. Power BI Dashboard and Reporting

The reporting layer is designed to communicate the processed airline data through interactive Power BI visuals. The assignment recommends KPI cards, filters, slicers, and visual sections for duration analysis, route performance, airline trends, and delay or anomaly insights.

A practical dashboard can begin with KPI cards showing the number of flights and average flight duration. A route visual can display traffic by source and destination. An airline visual can compare flight counts across airlines. Timing or anomaly visuals can highlight records that require further attention.

Slicers can be used to filter airline, source, destination, or other available dimensions. The dashboard should remain connected to the cleaned dataset so that the displayed metrics are based on standardized data rather than raw inconsistent values.

The final Power BI submission should include the PBIX report and screenshots as required by the case study.

17. Assumptions

The pipeline uses assumptions where the raw data alone cannot provide a complete business definition. These assumptions should be stated clearly so that users understand the limits of the analysis.

- Flight duration is derived from standardized departure and arrival timestamps.
- Cross-day flights are handled by assigning arrival to the next day when the time relationship indicates an overnight journey.
- Only fields supported by the supplied dataset are used for KPI calculations.
- Records that cannot support a reliable time-based calculation should not be treated as valid duration records.
- Business definitions such as a true operational delay should not be inferred beyond what the available fields can support.

18. Data Privacy and Governance

The supplied case study is based on flight-level operational information. The documented dataset fields do not require passenger-level analysis. If future versions of the source contain passenger personally identifiable information, those fields should be masked or removed before analytical use, following the assignment's data-quality and governance requirement.

Data governance also includes maintaining consistent schemas, documenting transformations, validating records, and controlling access to raw and curated data. Keeping the raw source separate from the transformed dataset helps preserve traceability and makes it possible to investigate unexpected KPI changes.

19. End-to-End Workflow

The complete workflow begins with source ingestion and ends with dashboard reporting. The sequence is important because KPI calculations depend on the quality of the upstream data.

- Load the raw airline file.
- Inspect schema, sample records, missing values, and duplicates.
- Clean duplicate and incomplete records according to the implemented rules.
- Standardize time and categorical fields.
- Handle overnight or cross-day flight timestamps.
- Calculate flight duration.
- Validate the transformed records and review anomalies.
- Aggregate data into business KPIs.
- Load or export the curated data for Power BI.
- Build interactive dashboard pages, filters, slicers, and KPI cards.

This workflow creates a clear chain from raw data to business reporting. Each stage can be independently reviewed, which improves maintainability and makes future extensions easier.

20. Key Outcomes and Insights

The completed implementation establishes a repeatable processing workflow for the ASG Airlines dataset. The main outcome is a cleaned and standardized flight dataset that is more suitable for analysis than the raw source.

The transformation of time fields enables reliable duration calculations, while cross-day handling prevents overnight flights from being incorrectly treated as negative-duration records. Duplicate and missing-value checks improve the reliability of flight counts and other aggregations.

The resulting structure supports the required KPI categories: average flight duration, route traffic, airline distribution, and anomaly-oriented analysis. These outputs provide the foundation for an interactive Power BI dashboard and for future extensions such as automated data-quality monitoring or scheduled ingestion.

21. Conclusion

The ASG Airlines case study demonstrates how a data engineering pipeline can convert raw operational data into trusted analytical information. The completed Google Colab implementation provides the processing layer for ingestion, cleaning, transformation, duration calculation, validation, and KPI preparation.

The most important technical consideration is accurate time handling, especially for flights that cross midnight. Combining data-quality checks with standardized time processing makes the resulting dataset more dependable for reporting.

The final solution can be extended toward a production-style architecture by adding scheduled ingestion, persistent storage, automated tests, monitoring, logging, and cloud services. For the current case study, the Python and Power BI workflow provides a clear and practical end-to-end implementation.

22. Dashboard Screenshot and Visual Interpretation

The final dashboard brings the project outputs together in one Power BI report page. It is organized so that high-level KPIs appear at the top, analytical charts occupy the middle area, filters are available on the left, and detailed passenger information is shown at the bottom.

The top KPI cards highlight Total Revenue, Average Flight Duration, and Airline Count. These cards are intended to give the report user an immediate summary before moving to detailed visuals.

The Flights by Airline chart provides an airline-level comparison. The Booking Status donut chart shows the distribution of booking states, while the Revenue by Payment Method chart compares the payment methods represented in the report.

The Top 10 Flight Routes chart supports route-level analysis. The Source and Destination slicers make it possible to narrow the analysis to selected locations. The Booking Status slicer provides another interactive filter.

The Passenger Details table contains passenger-related reporting information. In the visible dashboard, email and phone values are masked, which reduces direct exposure of contact information while retaining useful reporting context.

The screenshot also shows the Power BI development panes for Filters, Visualizations, and Data. These are part of the report-building environment and demonstrate the fields and visual configuration used to construct the dashboard.

Airline Booking Dashboard

Total Revenue
Average Flight Duration
Airline Count

13.74K

↑ 12.5% vs last period

3.06

↑ 8.2% vs last period

4

↑ 0.0% vs last period

Flights by Airline

Airline	Count of flight_id
SpiceJet	4

Booking Status

Status	Percentage
CONFIRMED	56%
CANCELLED	24%
PENDING	12%
UNKNOWN	8%

Revenue by Payment Method

Payment Method	Count of amount_numeric
NETBANKING	1.0
UPI	1.0

Booking Status

- ☐ CANCELLED
- ☒ CONFIRMED
- ☐ PENDING
- ☐ UNKNOWN

Top 10 Flight Routes

Route	Count of route
SJ058	1.0
SJ158	1.0
SJ171	1.0
SJ241	1.0
SJ421	1.0

Total Revenue

13.74K

Sum of amount_numeric

Source

- ☐ BLR
- ☐ BOM
- ☐ CCU
- ☒ DEL
- ☐ MAA

Destination

- ☐ BLR
- ☐ BOM
- ☐ CCU
- ☒ HYD
- ☐ MAA

Passenger Details

passenger_id	first_name	last_name	gender	email_masked	phone_masked	age
1	John	Smith	Male	joh***@mail.com	+91*****12	28
2	Emma	Johnson	Female	emna***@mail.com	+91*****45	34
3	Michael	Brown	Male	mike***@mail.com	+91*****78	42
4	Sophia	Davis	Female	sophia***@mail.com	+91*****21	27
5	William	Wilson	Male	wil***@mail.com	+91*****67	38

Filters

Search

Filters on this page

Add data fields here

Filters on all pages

Add data fields here

Visualizations

Build visual

Add data fields here

Values

Add data fields here

Drill through

Cross-report ☐ OFF

Keep all filters ☒

Add drill-through fields here

Data

Search

- ☒ arrival_time
 - ☐ Date Hierar...
 - ☐ > calculated_du...
 - ☐ Σ calculated_du...
 - ☐ corrected_arri...
 - ☐ Date Hierar...
 - ☐ > corrected_dur...
- ☒ departure_date
 - ☐ Date Hierar...
 - ☐ > departure_day...
 - ☐ departure_hour
- ☒ departure_time
 - ☐ Σ destination
 - ☐ duration
 - ☐ Σ duration_diff...
 - ☐ > existing_durat...
 - ☐ > flight_duratio...
 - ☐ Σ flight_id
 - ☐ > overnight_flag
 - ☐ > route
 - ☐ > source
- ☐ clean_passengers